DigitalLiteracyProjectReport

Name: Mandar Parkhe

RegistrationNo: 25BCE10093

Branch: B.Tech(CSE-Core)

Year: First Year (2025-2026)

Course Code: CSA2001

Date:29-03-2026

Project Report: AI-Powered SMS Spam Classifier

1. Executive Summary

This project demonstrates the application of Machine Learning in the domain of **Natural Language Processing (NLP)**. The objective is to build a classification model capable of distinguishing between "Ham" (legitimate messages) and "Spam" (unwanted/fraudulent messages). Using a **Multinomial Naive Bayes** approach, the system achieves high accuracy by analyzing the frequency of words within the messages.

---

2. Methodology & Workflow

The project follows a standard data science pipeline to transform raw text into a predictive model.

A. Data Acquisition & Cleaning

The dataset (typically the UCI SMS Spam Collection) consists of over 5,000 SMS messages.

- **Encoding:** The file is loaded using `latin-1` to handle special characters common in text data.
- **Filtering:** Redundant columns are dropped, leaving only `label` (target) and `message` (feature).
- **Label Encoding:** Categorical labels are converted to binary format:
  - **0:** Ham
  - **1:** Spam

B. Feature Extraction (Vectorization)

Computers cannot understand raw text. We use the **Bag of Words (BoW)** model via `CountVectorizer`.

- **Process:** It tokenizes the text, counts the frequency of each word, and creates a sparse matrix where each row represents a message and each column represents a unique word in the vocabulary.

C. The Algorithm: Multinomial Naive Bayes

The model uses the **Naive Bayes** algorithm, which is based on Bayes' Theorem:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

It is particularly effective for text classification because it assumes that the occurrence of one word is independent of another, making it computationally efficient and surprisingly accurate for spam detection.

---

3. Implementation Details

The model was built using **Python** and the **Scikit-Learn** ecosystem.

Component	Library/Tool
Language	Python 3.x
Data Manipulation	Pandas
Preprocessing	CountVectorizer
Model	MultinomialNB
Evaluation	Accuracy Score, Classification Report

---

4. Evaluation Metrics

The data is split into **80% Training** and **20% Testing** to ensure the model generalizes well to unseen data.

- **Accuracy:** Measures the percentage of total correct predictions.
- **Precision:** How many of the predicted "Spam" messages were actually spam? (Crucial to avoid filtering out important "Ham" emails).
- **Recall:** How many of the actual spam messages did the model successfully catch?

---

5. Sample Prediction Case

The system includes a real-time testing function. For example:

Input: *"Congratulations! You've won a $1,000 Walmart gift card. Click here to claim."* >
Result: `SPAM` □

---

6. Conclusion & Future Enhancements

The project successfully builds a lightweight, fast, and accurate spam filter. While the current model is highly effective, future improvements could include:

1. **Stop-word Removal:** Removing common words like "the", "is", and "at" to focus on more meaningful keywords.
2. **TF-IDF Vectorization:** Using Term Frequency-Inverse Document Frequency to penalize words that appear too frequently across all messages.
3. **Stemming/Lemmatization:** Reducing words to their root form (e.g., "running" to "run") to consolidate the vocabulary.